MANCHESTER 1824
The University of Manchester

A Modular Real-Time Haptic Rendering System Integrating Physics-Based Simulation and OpenGL Visualisation

A dissertation submitted to The University of Manchester for the degree of

Bachelor of Engineering in Mechatronic Engineering

in the Faculty of Science and Engineering

Year of submission

2026

Student ID

11349510

School of Engineering

Abstract

This is abstract text.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Declaration of originality

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright statement

i The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the "Copyright") and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made *only* in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the "Intellectual Property") and any reproductions of copyright works in the thesis, for example graphs and tables ("Reproductions"), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420`), in any relevant Thesis restriction declarations deposited in the University Library, The University Library's regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University's policy on Presentation of Theses.

Acknowledgments

Acknowledgments go here.

1 Introduction

1.1 Background and Motivation

Robotic manipulators are increasingly used in domains such as industrial automation, teleoperation, and robot-assisted surgery, where safe and effective interaction with physical environments is essential. In many of these contexts, operator training plays a critical role in determining task quality, safety, and efficiency. Although visual feedback provides important global information about the workspace, it is often insufficient for developing fine motor control, accurate force regulation, and intuitive understanding of contact conditions.

Haptic interfaces address this limitation by enabling bidirectional physical interaction between a user and a virtual or remote environment. By rendering forces back to the operator, such systems can communicate contact, constraint, and interaction information that is difficult to convey visually alone [1]. In robotic manipulation tasks, this is particularly valuable for training applications, where users must learn not only where to move, but also how to regulate force and respond to contact conditions. Prior work has shown that haptic feedback can improve task performance and reduce excessive applied force, particularly for less experienced users [2].

Despite these benefits, access to haptic systems remains limited. Commercial haptic interfaces provide mature and high-performance force feedback, but they are typically expensive and frequently tied to proprietary hardware and software ecosystems. This can restrict accessibility, reduce architectural transparency, and make it difficult for researchers and students to experiment with alternative control strategies, custom simulation environments, or modified hardware configurations. As a result, there is strong practical motivation for low-cost and modular haptic platforms that are suitable for prototyping, experimentation, and robotics training research.

At the same time, achieving stable and convincing haptic interaction is technically challenging. Real-time haptic rendering commonly requires update rates on the order of 1 kHz together with low latency and carefully designed control structure in order to render contact stably and responsively [3]. The stability of haptic interaction is particularly sensitive to delay in the feedback loop, since communication and computation delays introduce phase lag and reduce the maximum achievable virtual stiffness [4]. Consequently, the design of a practical haptic system is not simply a matter of adding actuators to an input device: it requires coordinated consideration of mechanical design, embedded control, communication architecture, simulation coupling, and force rendering strategy.

1.2 Problem Definition

This dissertation addresses the problem of how to realise a *low-cost*, *modular*, *real-time haptic interface system* suitable for robotic manipulation training and experimentation, while explicitly accounting for the practical constraints that arise in implementation.

The specific challenge is not merely to construct a haptic input device, but to integrate multiple subsystems into a stable bidirectional interaction loop: user motion must be sensed at the device, transmitted to a host system, interpreted within a simulated environment, converted into interaction forces, mapped back into actuator commands, and rendered to the user in real time. In practice, each stage of this loop introduces constraints. Limited actuator capability affects the achievable force output, communication delay limits responsiveness and stable virtual stiffness, and software architecture determines whether the system remains extensible and maintainable.

Existing work demonstrates the value of haptic feedback and the feasibility of low-cost haptic hardware, but there remains a practical gap between simple hardware demonstrations and fully integrated systems that combine low-cost actuation, real-time communication, physics-based simulation, and explicit treatment of latency and stability. Educational and open-source devices have shown that affordable haptic hardware is possible [5], [6], and low-cost 2-DOF academic systems have also been demonstrated **v**. However, comparatively less emphasis is placed on complete end-to-end architectures in which hardware, embedded control, host-side simulation, rendering, and communication are developed together and evaluated as an integrated real-time system.

The problem addressed in this project is therefore defined as follows: *to design*, *implement*, *and evaluate a low-cost haptic rendering system that is sufficiently modular for experimentation*, *sufficiently responsive for real-time interaction*, *and sufficiently stable to demonstrate practical bidirectional haptic coupling with a simulated robotic environment*.

1.3 Aim and Scope of This Dissertation

The aim of this project is to design, implement, and evaluate a modular low-cost haptic rendering system integrating a two-degree-of-freedom input device with a custom real-time simulation and visualisation framework.

The project focuses on demonstrating the feasibility of an end-to-end architecture rather than on

producing a commercial-grade haptic product. More specifically, the dissertation is concerned with:

- the design and construction of a low-cost actuated 2-DOF haptic device,
- the development of embedded firmware for sensing, communication, and actuator control,
- the implementation of a host-side software framework supporting simulation, haptic rendering, and visualisation,
- the integration of these components into a real-time bidirectional interaction loop, and
- the experimental evaluation of communication performance, timing behaviour, stability, and qualitative interaction characteristics.

The scope of the work is intentionally limited. The dissertation does not attempt to demonstrate fully calibrated force output, high-fidelity commercial-level rendering, or a formal user study of training effectiveness. In addition, while the device was designed as a 2-DOF haptic interface, final hardware validation was performed with force feedback available on only one joint due to actuator failure during development. Accordingly, the main contribution of the work lies in the *system-level design, integration, and evaluation of a modular real-time prototype*, together with analysis of the practical factors that limit its performance.

This framing is important because the value of the dissertation lies not only in demonstrating that low-cost haptic interaction is possible, but also in identifying the architectural and hardware constraints that dominate performance in practice. As the results later show, the principal limitation of the implemented system is not host-side computational speed, but communication latency and the resulting constraint it imposes on achievable haptic stiffness and responsiveness.

1.4 Objectives

To achieve this aim, the project pursued the following objectives:

- to design and construct a 2-DOF haptic device using low-cost brushless actuators, magnetic encoders, and lightweight fabricated structural components;
- to implement embedded firmware for encoder acquisition, actuator control, and bidirectional packet-based communication;
- to develop a modular host-side software architecture supporting world management, physics simulation, haptic rendering, and graphical visualisation;

- to implement proxy-based contact rendering and Jacobian-based force-to-torque mapping for haptic feedback generation;
- to evaluate the resulting system in terms of communication behaviour, closed-loop timing, stability, safety, and qualitative force-feedback performance; and
- to assess the practical limitations of the architecture and identify directions for future improvement.

1.5 Contributions of This Work

The work presented in this dissertation makes the following principal contributions:

- the design and implementation of a low-cost actuated haptic interface prototype based on a 2-DOF mechanical architecture;
- the development of a custom modular host-side software framework integrating haptic rendering, rigid-body simulation, and OpenGL-based visualisation;
- the implementation of a bidirectional real-time interaction pipeline linking the physical device to the simulated environment;
- an experimental evaluation showing that the system can achieve stable real-time interaction at high update rates, while also revealing communication latency as the dominant limitation on performance; and
- a critical analysis of the trade-offs between cost, modularity, stability, and achievable haptic fidelity in a student-scale prototype system.

2 Literature review

2.1 Haptic Interfaces for Robotics

Haptic interfaces enable bidirectional interaction between a user and a virtual or remote environment, providing force feedback that complements visual information and supports the development of fine motor control and contact awareness [1]. This is particularly important in robotic manipulation and training contexts, where users must regulate both motion and applied force. Prior work shows that haptic feedback can significantly improve task performance, especially for novice users [2].

Commercial haptic devices such as the 3D Systems *Geomagic Touch* and *Touch X*, the *Force Dimension* product range, and the Haply *Inverse3 X* provide high-fidelity kinesthetic feedback and mature software ecosystems. However, these systems are typically expensive and often rely on proprietary hardware and software, which can limit accessibility and flexibility [7]. For example, the Geomagic Touch and Touch X are publicly listed at approximately US$3,400 and US$13,400 respectively [8], [9], placing them well above the cost of low-cost academic prototypes.

2.2 Haptic Rendering Techniques

Real-time haptic rendering has been extensively studied, with particular focus on contact-based interaction models. Two widely adopted approaches are virtual coupling and the virtual proxy (or "god-object") method introduced by Zilles and Salisbury and later extended by Ruspini *et al.* [3], [10].

In these approaches, the device state is coupled to a constrained proxy within the virtual environment, enabling stable rendering of unilateral contact and geometric constraints. Reliable performance typically requires high update rates on the order of 1 kHz, together with accurate tracking of device state and consistent mapping from Cartesian interaction forces to actuator torques [3].

2.3 Stability in Haptic Systems

Stable haptic interaction imposes strict requirements on latency, control architecture, and numerical implementation. Adams and Hannaford demonstrated that virtual coupling can provide stability guarantees by decoupling device dynamics from the virtual environment under passive interaction assumptions [4].

These results highlight the sensitivity of haptic systems to delay and discrete-time effects, and motivate the use of control structures that explicitly manage dynamic coupling and feedback timing. In practical systems, communication latency and asynchronous computation introduce additional challenges that must be addressed at the architectural level.

2.4 Low-Cost Haptic Hardware

Advances in low-cost components, including 3D-printed structures, microcontrollers, and magnetic encoders, have enabled the development of accessible haptic devices using widely available hardware. This has led to increasing interest in open and modular platforms for research and education [1].

The *Hapkit* platform developed by Martinez *et al.* demonstrates that functional haptic devices can be realised at very low cost (approximately US$50) using 3D-printed components [5]. Subsequent work extended this approach to modular multi-degree-of-freedom configurations, including both pantograph and serial mechanisms [6].

Similarly, Lavatelli *et al.* presented an open-source low-cost 2-DOF haptic device, demonstrating that multi-degree-of-freedom actuation can be achieved within an academic budget while maintaining useful interaction capability [11].

However, much of this work focuses primarily on hardware design or educational platforms. Comparatively less emphasis is placed on fully integrated systems that combine low-cost hardware with real-time communication, simulation coupling, and extensible software architectures.

2.5 Summary of Literature

The literature establishes that haptic feedback improves performance in robotic manipulation tasks and provides well-developed methods for stable force rendering. Low-cost hardware platforms have also demonstrated that accessible haptic devices are feasible.

However, there remains a gap between simplified hardware demonstrations and fully integrated real-time systems that combine low-cost actuation, transparent communication, simulation, and extensible software design. Addressing this gap motivates the development of a modular, low-cost haptic system capable of supporting real-time bidirectional interaction with a simulated robotic environment.

3 System Architecture

3.1 System Overview

The proposed system implements a bidirectional haptic interaction loop consisting of four principal layers: the haptic device hardware, embedded control firmware, host-side simulation and haptic rendering, and graphical visualisation. A conceptual overview of the signal and force flow is shown in Fig. 1.

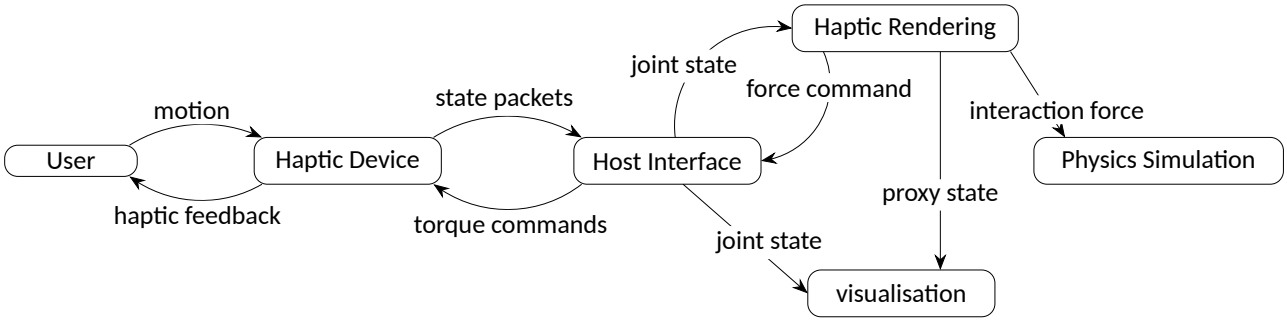


Fig. 1. High-level architecture of the proposed haptic system. User motion is measured by the haptic device and transmitted to the host interface. The host distributes state information to the haptic rendering and visualisation subsystems, while the haptic rendering module computes interaction forces, updates the simulation, and generates feedback commands that are returned to the device as actuator torque commands.

User motion is measured at the device joints and transmitted to the host in real time. The host computes interaction forces based on the virtual environment state, maps these to joint torque commands, and returns them to the device actuators, forming a closed-loop interaction.

To support stable real-time operation, time-critical control tasks are executed on the embedded system at high frequency, while simulation, rendering, and visualisation are performed asynchronously on the host.

3.2 Design Constraints and Assumptions

The system is designed to prioritise real-time stability, modularity, and reproducibility within the constraints of a low-cost prototype.

Stable haptic interaction requires high update rates (approximately 1 kHz) and low end-to-end latency. Accordingly, the architecture assumes high-frequency embedded torque control and asynchronous host-side computation.

Mechanical and control limits are selected to ensure safe operation while maintaining sufficient force capability for meaningful interaction.

3.3 Hardware Architecture

The haptic interface is implemented as a planar two-degree-of-freedom mechanism with orthogonal revolute joints. Each joint is actuated by a brushless motor operating in torque control mode and instrumented with absolute angular position sensing.

The structure is designed to be lightweight and back-drivable to minimise reflected inertia and improve transparency.

3.4 Embedded Control Architecture

The embedded system performs real-time actuator control, sensor acquisition, and communication with the host. Joint states are streamed to the host at a fixed rate, while torque commands are received and applied continuously.

Safety mechanisms, including torque limiting and communication watchdogs, are implemented to ensure predictable behaviour under fault conditions.

3.5 Communication Architecture

Communication between the embedded controller and host is implemented using a packet-based interface exchanging joint state and torque command data.

The protocol is designed to support deterministic timing and low overhead suitable for real-time haptic interaction.

3.6 Host Software Architecture

The host system is implemented as a modular multi-threaded application comprising subsystems for world management, physics simulation, haptic rendering, device communication, and visualisation.

These subsystems operate at different rates and exchange data through structured communication channels, allowing high-frequency haptic computation to be maintained independently of slower processes such as rendering.

3.7 Haptic Rendering Paradigm

Interaction forces are generated using a proxy-based rendering approach in which the device position is coupled to a constraint-based proxy within the virtual environment. This enables stable rendering of contact interactions under discrete-time computation.

3.8 Force Transmission to the Device

Cartesian interaction forces are mapped to joint torques using the Jacobian transpose formulation:

$$\tau = J^T(q)F \tag{1}$$

where τ is the joint torque vector, $J(q)$ is the manipulator Jacobian, and F is the rendered force.

3.9 Visualisation Layer

A graphical visualisation subsystem provides real-time feedback of the simulated environment. Rendering is decoupled from the haptic loop to prevent graphical latency from affecting interaction stability.

4 System Implementation

This section describes the realised haptic system implementation, covering the physical device, embedded control and communication, and the host-side software stack used for rendering, simulation, and force feedback.

The system prioritises modularity and deterministic timing: each major subsystem runs on its own thread and exchanges data through explicit message channels. This design reflects a trade-off be-

tween cost, achievable force fidelity, and system stability, with particular emphasis on maintaining stable interaction under non-negligible communication latency.

4.1 Mechanical and Embedded Platform

The physical interface is a planar two degree-of-freedom serial mechanism designed for low-cost fabrication and straightforward maintenance. A two-link layout was selected to provide a usable workspace while keeping actuator count, wiring complexity, and control overhead manageable.

The structure is primarily 3D printed, with each joint supported by steel shafts and rolling element bearings to reduce friction and improve backdrivability. Actuation is provided by brushless gimbal motors (T-Motor GB2208). The base joint incorporates a GT2 belt reduction stage (approximately 3:1) to increase available output torque, while the distal joint is direct-drive to minimise mechanical complexity.

An adjustable idler pulley is integrated into the belt stage to enable tensioning and reduce backlash. Despite this, non-ideal behaviour such as compliance and occasional slip under high load was observed, as discussed in Section 7.

To ensure accurate joint angle measurement, absolute magnetic encoders (AS5048) are mounted directly on the joint axes and interfaced via SPI. This is particularly important for the belt-driven joint, where transmission compliance would otherwise introduce discrepancies between motor-side and output angles.

For the belt-driven actuator, an additional AS5600 magnetic encoder is mounted on the motor shaft and interfaced via I^2C. This provides rotor position feedback required for field-oriented control (FOC), while the joint encoder provides accurate output angle measurement.

Embedded control runs on an STM32 Nucleo-G401RE platform with SimpleFOC Mini drivers and the SimpleFOC software stack [12]. Both actuators operate in torque-mode FOC [13]. A communication watchdog is implemented in firmware: if valid host commands are not received within the timeout window, output torques are forced to zero.

Due to hardware constraints, no direct force sensing or current measurement was implemented; as a result, torque output is estimated rather than directly measured. Additionally, one actuator failed during development, meaning final hardware validation was conducted using single-axis actuation, although both joints remained available for sensing.

PLACEHOLDER FIGURE

Mechanical diagram
(links, joints, belt reduction, encoders)

Fig. 2. Mechanical layout of the haptic device.

4.2 Communication Protocol and Device Interface

Host–device communication uses fixed-format binary packets over a USB virtual serial link at high update rate (target 1 kHz exchange). Two packet families are used:

- device state packets (firmware $\rightarrow$ host), containing sequence number, MCU timestamp, and joint angles
- torque command packets (host $\rightarrow$ firmware), containing command sequence, referenced state sequence, and commanded joint torques

Sequence numbers enable detection of dropped or out-of-order packets, while MCU timestamps provide a reference for timing analysis and latency measurement.

Both packet types include a two-byte header for stream re-synchronisation and a checksum for integrity checks.

On the host, serial I/O and protocol handling are split across two layers:

- *SerialLink*: low-level COM read/write abstraction
- *DeviceAdapter*: packet parsing, forward kinematics, Cartesian force-to-joint torque mapping, and message-bus integration

The device adapter runs in a dedicated 1 kHz loop. At each cycle, it retains the newest valid state packet, publishes tool pose and velocity to the haptic engine, consumes the latest wrench command, computes joint torques using the planar Jacobian transpose relation $\tau = J^T F$, and transmits the resulting torque command packet.

This ensures that control decisions are always based on the most recent available state, avoiding backlog-induced latency and improving responsiveness under high update rates.

4.3 Host Software Architecture

PLACEHOLDER FIGURE

System architecture diagram
(threads + message channels)

Fig. 3. Overview of the host-side software architecture.

The host application is implemented as a modular multi-threaded C++ system. Major responsibilities are partitioned into the renderer, world manager, physics engine, haptic engine, and device adapter. Startup composition occurs in `main.cpp`, where channels are created, subsystems are constructed, and execution threads are launched.

Inter-thread communication is implemented through a message bus with two channel types:

- queue channels for discrete commands/events (e.g., world edits, wrench messages)
- snapshot channels for latest-value shared state (e.g., world snapshots)

This separation allows command traffic to be drained in batches while continuous state can be accessed with low latency.

4.3.1 World and Rendering Subsystems

WorldManager is the authoritative owner of virtual scene state. Objects store identity, geometry reference, pose, appearance, role labels, and physics properties. Subsystems do not mutate world data directly; instead, they publish commands which are applied centrally by the world manager.

Rendering is handled by an OpenGL subsystem running on the main thread. Each frame reads the latest world snapshot, resolves geometry and mesh handles, composes model transforms from ob-

ject pose, and renders via shared shader and mesh abstractions. An ImGui interface provides runtime controls for object editing and parameter tuning; UI edits are emitted as world commands to preserve ownership boundaries.

4.3.2 Haptic Rendering Engine

The haptic engine executes at 1 kHz in a dedicated loop. At each cycle, it reads the latest world snapshot and newest tool state, performs contact queries, computes force output, and publishes results to device, physics, and visualisation channels.

Contact detection is performed using signed distance field (SDF) queries, with penetration depth and surface normals obtained from the local distance field gradient [14]. For each candidate object, the tool position is transformed into object-local coordinates and queried against that object's SDF. If penetration is detected (negative signed distance), the tool point is projected to the surface to define the proxy position; otherwise, the proxy follows the tool in free space.

Force output is generated using a proxy-based virtual coupling model, in which a spring–damper relationship is defined between the device position and a contact-constrained proxy [4]:

$$\mathbf{F} = K\left(\mathbf{x}_p - \mathbf{x}_t\right) + D\left(\mathbf{v}_p - \mathbf{v}_t\right)$$

where $\mathbf{x}_p, \mathbf{x}_t$ are proxy and tool positions and $\mathbf{v}_p, \mathbf{v}_t$ are corresponding velocities. In the implemented configuration, $K = 500\,\mathrm{N\,m^{-1}}$, $M = 0.02\,\mathrm{kg}$, and

$$D = 0.7 \cdot 2\sqrt{KM}.$$

This follows the standard virtual coupling formulation, although the present implementation does not explicitly enforce passivity-based design constraints. In particular, the proxy state is recomputed directly from the current tool position and contact geometry at each timestep, rather than being evolved as a constrained dynamic system. As a result, the implementation captures the force-generation behaviour of virtual coupling but does not realise the full passivity-based formulation described by Adams and Hannaford.

To maintain safe interaction, force magnitude is clamped to a maximum of 31 N before publication.

PLACEHOLDER FIGURE

Proxy-based haptic rendering
(tool, proxy, surface, force vector)

Fig. 4. Proxy-based contact rendering using virtual coupling.

4.3.3 Physics Simulation Integration

Rigid-body dynamics are simulated using NVIDIA PhysX. The physics subsystem runs on its own simulation thread and consumes wrench messages from the haptic loop. Incoming forces are converted to impulses over the effective timestep,

$$\mathbf{J} = \mathbf{F}\,\Delta t,$$

and applied at the contact point to the corresponding actor.

A fixed-step accumulator scheme is used to decouple simulation stability from variable external loop timing. When world topology or physics properties change, actors are rebuilt from the latest authoritative world snapshot. After each simulation update, dynamic poses are written back to the world manager so subsequent rendering and haptic queries operate on consistent state.

4.4 Logging and Communication Validation

To avoid introducing disk latency into the 1 kHz control path, runtime logging is performed asynchronously. The device thread publishes structured timing and state messages to dedicated logging channels, and a background logger thread drains these channels into memory.

At shutdown, logs are exported to two CSV files:

- `device_timing.csv`: matched state-command timing records for closed-loop control cycles, including host-side timestamps associated with packet parsing, tool-state publication, wrench

consumption, and torque command transmission

- `device_state_log.csv`: a record of every valid parsed incoming state packet, including host receive timestamps, packet sequence identifiers, MCU timestamps, and measured joint angles

A separate communication test executable (`serial_tests`) was also implemented to characterise serial performance independently of the full application. It supports two operating modes: a packet rate and integrity mode for analysing streamed device packets, and a ping–echo mode for measuring round-trip latency. In both modes, incoming serial data are processed using a buffered parser with explicit packet header search, checksum validation, and byte-wise re-synchronisation to ensure robustness to fragmented or corrupted input. Results are exported to machine-readable CSV files for offline analysis.

5 Experimental Methodology

This chapter defines the test procedures used to evaluate communication performance, closed-loop timing behaviour, simulation fidelity, safety behaviour, and hardware interaction characteristics of the proposed haptic rendering system.

The chapter focuses on what was tested and how measurements were collected; quantitative outcomes are reported in Chapter 6 and interpreted in Chapter 7.

5.1 Communication Performance Tests

Communication performance was characterised using the dedicated `serial_tests` utility described in Section 4.4. The tool supports two modes: a packet rate and integrity mode for analysing streamed device packets, and a ping–echo mode for measuring round-trip latency. Both modes export CSV files for offline statistical analysis.

Tests were conducted across target transmission rates from 100 Hz to 2000 Hz to assess the effect of increasing packet frequency on reliability and timing performance.

5.1.1 Packet Integrity and Rate Tests

Packet integrity and communication timing were evaluated using the rate-test mode of the communication test utility.

For each target transmission frequency, the firmware was configured to stream fixed-format device state packets at a constant rate. Tests were performed at nominal frequencies of 100 Hz, 250 Hz, 500 Hz, 1000 Hz, 1250 Hz, and 2000 Hz.

For each test condition, the host continuously received and parsed incoming packets over the USB virtual COM interface. The recorded performance metrics included valid packet count, dropped packets inferred from sequence gaps, out-of-order packets, host-observed inter-arrival time, effective receive rate, and inter-arrival jitter.

All results were exported to CSV format for offline statistical analysis and visualisation.

5.1.2 Round-Trip Latency Test

Round-trip latency was evaluated using a ping–echo communication test.

In this test, the host transmitted timestamped ping packets to the embedded controller, which immediately returned an echo packet containing the original sequence number and timestamp.

Tests were performed using transmission intervals of 1 ms, 2 ms, and 10 ms. For each test, the evaluated metrics included mean, median, 95th percentile, and 99th percentile RTT, along with minimum and maximum RTT values and timeout counts.

These results were used to characterise communication latency and timing consistency under different transmission loads.

5.1.3 End-to-End Host-Side Timing Test

In addition to the standalone communication tests, end-to-end timing of the host-side haptic pipeline was evaluated using the runtime logging framework described in Section 4.4. Logs were captured during normal closed-loop operation and exported to `device_timing.csv` and `device_state_log.csv` for offline analysis in MATLAB.

The evaluation included host-side end-to-end latency between state-packet parsing and torque command transmission, intermediate pipeline delays between communication, haptic processing, and transmission stages, matched state-command sequence consistency, parsed state-packet inter-arrival timing, sequence gap statistics, and estimated parsed packet rate with MCU-side timing trends.

Because the host and microcontroller clocks were not synchronised, the analysis was restricted to host-side timing differences and trend-based interpretation of MCU timestamps. Absolute one-way latency between host and device could therefore not be inferred directly.

5.2 Simulation Validation

Simulation validation tests were designed to assess both the correctness of the haptic rendering model in isolation and the stability of the integrated system during interaction.

Two complementary evaluations were performed. First, the behaviour of the virtual coupling and contact model was assessed by observing the computed force output under controlled interaction conditions. Second, the full system, including the physical device, communication pipeline, and actuator feedback, was evaluated during sustained user interaction.

5.2.1 Virtual Wall Test

A virtual wall test was performed to validate the basic unilateral contact behaviour of the haptic rendering engine. A planar contact boundary was defined in the virtual environment, and the device end-effector was manually moved toward and against the wall during repeated trials.

This test verified that the proxy-based rendering method prevented penetration of the constrained proxy through the wall surface while allowing the unconstrained device position to continue moving, thereby generating a restoring virtual coupling force.

The evaluation examined the absence of significant rendered force outside the wall, the increase in force with penetration depth, and qualitative agreement with the implemented spring–damper coupling model.

5.2.2 Constrained Motion Along a Virtual Surface

The system was further evaluated for its ability to support constrained motion along a virtual surface.

During this test, the end-effector was brought into contact with a planar wall and allowed to move along the surface while maintaining contact. This behaviour arises naturally from the proxy-based rendering method, where motion normal to the surface is resisted while tangential motion remains unconstrained.

The evaluation assessed the ability to maintain contact during motion, the absence of resistance to tangential movement, stability during sustained sliding interaction, and smooth transitions between free-space and constrained motion.

5.3 Safety Validation

Safety validation tests ensured that the system remained bounded and behaved predictably under both normal and fault conditions. These tests focused on verifying torque saturation, rate limiting, and watchdog-based fail-safe behaviour.

5.3.1 Torque Saturation Test

A torque saturation test verified that actuator command limits were correctly enforced across both host and embedded layers.

Force outputs were limited on the host side to a predefined maximum, and the embedded controller applied an additional clamp to ensure commands remained within safe bounds. Testing confirmed that:

- host-side force commands were correctly limited
- transmitted torque commands were further bounded by the embedded controller
- actuator output remained within safe and predictable limits

5.3.2 Rate Limiting Test

A rate limiting test evaluated the effect of constraining the rate of change of commanded torque at the embedded controller level.

A slew-rate limiter was implemented to reduce high-frequency oscillations and improve stability during rapid contact transitions. Testing verified that:

- rapid command changes were smoothed into bounded torque ramps
- high-frequency oscillations were reduced during contact events
- overall system stability improved during interaction

5.3.3 Watchdog Timeout Test

A watchdog timeout test verified safe system behaviour under communication loss by interrupting the incoming command stream during operation.

The embedded controller response confirmed that:

- the watchdog triggered reliably when command updates ceased
- commanded torque was removed within the intended timeout window
- the device returned to a safe passive state

5.4 Hardware Validation

Hardware validation assessed the behaviour of the physical actuation system and overall device interaction, focusing on qualitative evaluation of responsiveness and stability.

5.4.1 Actuator Torque Response Characterisation

Basic actuator response behaviour was evaluated using standalone motor testing prior to system integration. Step changes in commanded torque were applied, and the resulting mechanical response was assessed qualitatively.

The evaluation examined responsiveness to torque changes, presence of oscillatory behaviour, smoothness of motion, and repeatability across repeated inputs. This represents qualitative validation rather than precise quantitative characterisation.

5.4.2 Stable Contact Test

A stable contact test evaluated the behaviour of the full closed-loop system during sustained interaction with a virtual surface.

The end-effector was brought into contact with a virtual object and maintained in contact while observing the resulting force feedback behaviour. The evaluation assessed the ability to maintain continuous contact without instability, the presence of oscillation or chatter, consistency of the rendered resistive force, and overall usability of the interaction.

6 Results

This chapter reports the measured outcomes of the Chapter 5 tests. It presents communication timing, simulation behaviour, safety outcomes, and hardware interaction observations.

Interpretation of these findings and their design implications is deferred to Chapter 7.

6.1 Communication Performance

Communication performance was evaluated using standalone serial tests and runtime logging of the integrated haptic application.

6.1.1 Packet Rate and Integrity Results

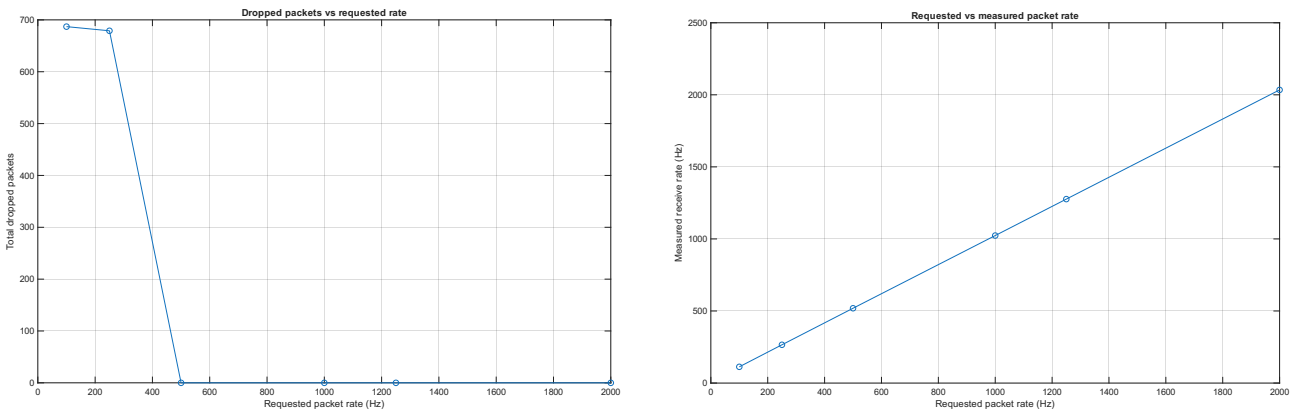
Packet rate testing showed that communication performance remained stable over a wide range of requested transmission frequencies, although the behaviour was not uniform across all tested rates.

At lower requested rates of 100 Hz and 250 Hz, packet loss was observed, with drop fractions of approximately 20.3% and 8.5% respectively. In contrast, for requested rates of 500 Hz and above,

no dropped packets were observed in the recorded data, as shown in Fig. 5(a), where packet loss occurs only at the lowest transmission rates. Out-of-order packets were extremely rare across all higher-rate tests, with only a single out-of-order event recorded in each case.

Interestingly, packet loss was more pronounced at lower transmission rates. This may be attributed to buffering and scheduling effects in the host or serial driver, where lower rates reduce the likelihood of continuous buffer flushing.

The measured receive rate tracked the requested transmission rate closely across the full test range. As shown in Fig. 5(b), the relationship between requested and measured rate was approximately linear, indicating that the communication system maintained throughput proportional to the commanded transmission frequency.



(a) Dropped packets **(b)** Rate tracking

Fig. 5. Communication performance across transmission rates. (a) Total dropped packets as a function of requested rate, showing packet loss only at low frequencies. (b) Measured receive rate versus requested transmission rate, demonstrating approximately linear tracking.

Inter-arrival jitter, defined in Section 5.1.1 as the standard deviation of packet inter-arrival time, was present in all tests, with standard deviation decreasing as the requested rate increased, as shown in Fig. 6. However, percentile analysis revealed that occasional large timing excursions persisted even at higher rates, indicating the presence of intermittent latency spikes. This suggests that while average jitter decreases with increasing rate, worst-case timing behaviour remains non-negligible, which is relevant for haptic stability.

Overall, the serial link supported packet rates at or above 1 kHz with negligible packet loss in the tested configuration.

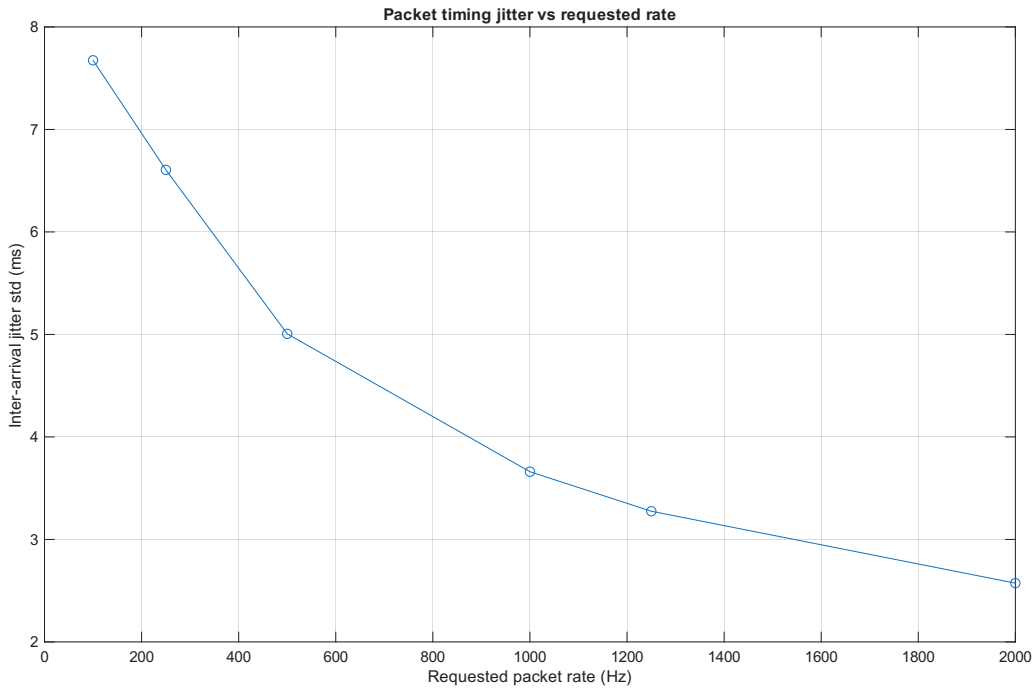


Fig. 6. Standard deviation of packet inter-arrival time as a function of requested transmission rate. Jitter decreases with increasing rate, although occasional large timing excursions remain present.

6.1.2 Round-Trip Latency Results

Round-trip latency results were highly consistent across all tested ping intervals. For nominal transmission intervals of 1 ms, 2 ms, and 10 ms, the success rate was 100% in all cases, with no timeouts observed over 1000 samples per test.

The mean measured round-trip latency remained approximately constant across all three test conditions, with values of 15.60 ms, 15.64 ms, and 15.67 ms respectively. Similarly, the 95th and 99th percentile RTT values were tightly grouped around 16.2–16.4 ms, indicating stable and predictable timing behaviour across repeated measurements. This behaviour is further illustrated in Fig. 7, where the cumulative distributions for all three tests overlap closely and rise sharply at approximately 15–16 ms, indicating that RTT is largely independent of the selected ping interval.

The standard deviation of RTT remained below 0.52 ms in all cases, suggesting relatively low variability in round-trip timing. Minimum and maximum observed RTT values showed occasional larger deviations from the typical latency range, particularly in the 1 ms test, but these did not significantly affect the overall distribution.

These results indicate that RTT was dominated by a largely fixed delay component and changed little with ping interval. The measured delay remained predictable due to low variability and zero observed timeouts.

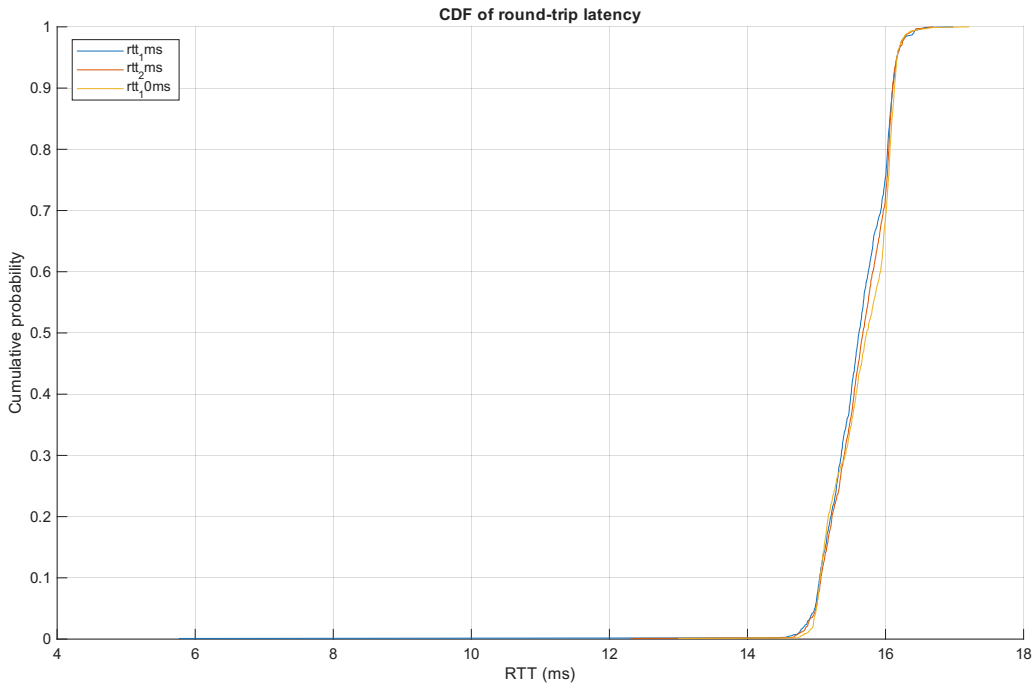


Fig. 7. Cumulative distribution functions of measured round-trip latency for nominal ping intervals of 1 ms, 2 ms, and 10 ms. The distributions overlap closely and rise sharply at approximately 15–16 ms, indicating that round-trip latency is dominated by a largely fixed delay component rather than by the selected ping interval.

6.1.3 End-to-End Host-Side Timing Results

End-to-end timing behaviour of the integrated haptic pipeline was evaluated using runtime logging of matched state-command cycles and raw parsed state packets.

Analysis of `device_timing.csv` showed that all recorded control cycles were successfully matched, with a matched fraction of 1.000, indicating consistent correspondence between received state packets and transmitted torque commands.

The host-side end-to-end latency, defined as the time between state-packet parsing and completion of torque command transmission, was found to be low. The mean latency was 0.299 ms, with a median of 0.104 ms. The 95th percentile latency was 0.546 ms, and the 99th percentile was 2.04 ms, with a maximum observed latency of 3.35 ms. As shown in Fig. 8, the distribution was concentrated at low values, with prominent peaks near 0 ms and around 0.5 ms, and only a limited number of higher-latency events.

Breakdown of intermediate pipeline delays showed that the majority of this latency was associated with serial transmission. The mean transmission duration was 0.249 ms, while the delay associated with publishing tool state and processing the haptic pipeline remained small, with mean values of 0.024 ms and 0.0044 ms respectively. This indicates that host-side computation and message-

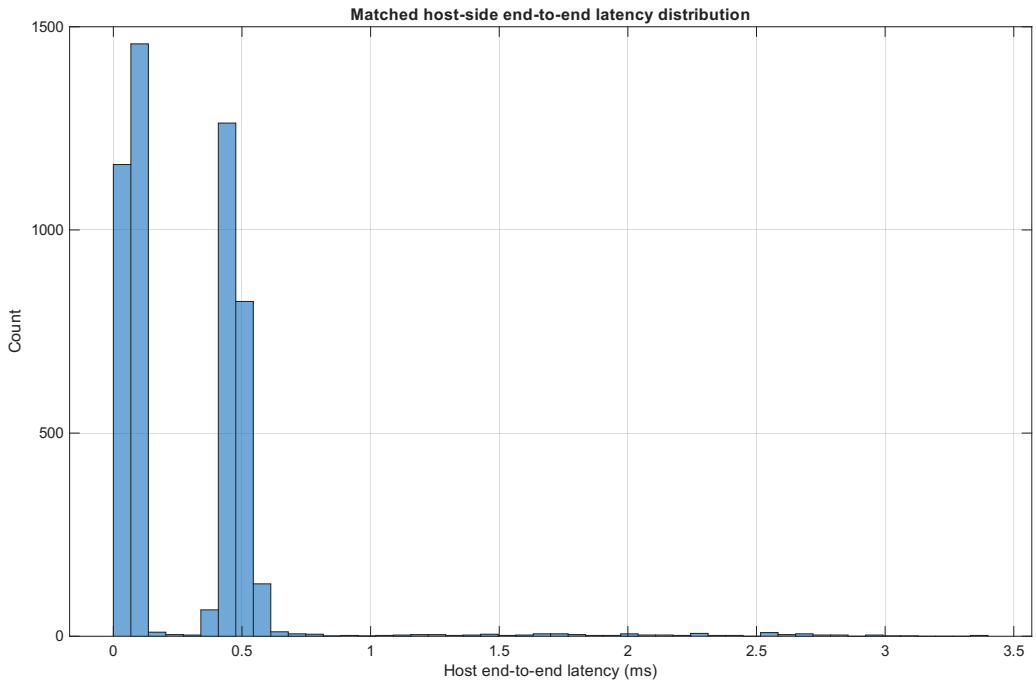


Fig. 8. Distribution of matched host-side end-to-end latency, measured from state-packet parsing to completion of torque command transmission. The latency is concentrated at low values, with prominent peaks near 0 ms and around 0.5 ms, and only a small number of higher-latency outliers.

passing overhead were negligible relative to communication time.

Sequence analysis showed that command sequence progression remained consistent, with a mean command sequence gap of 1.14 and a maximum of 6, indicating that command updates were generally applied sequentially with minimal skipping. In contrast, the matched state sequence gap was significantly larger, with a mean of 33.45. This reflects the fact that the control loop operated on a subset of the available incoming state packets rather than processing every parsed packet. Here, sequence gap denotes the difference between consecutive sequence identifiers, such that a value of 1 corresponds to consecutive packets or commands and larger values indicate skipped intermediate sequence numbers.

A total of 54 161 valid state packets were parsed during the test, compared to 5 048 matched control cycles, confirming that the control loop operated on a subset of the incoming packet stream rather than consuming every parsed packet. The estimated parsed packet rate, computed from the mean host-side inter-arrival time, was approximately 1006 Hz, which is close to the expected 1 kHz transmission rate.

Analysis of host-side parsed-packet inter-arrival timing revealed significant variability. The mean inter-arrival time was 0.994 ms, but the median was only 0.0016 ms, reflecting that many packets were processed in immediate succession within bursts. The large difference between mean and

median inter-arrival time indicates that packets were frequently processed in rapid bursts following periods of delay, rather than arriving at uniform intervals. This behaviour is illustrated in Fig. 9, where a dominant peak near 0 ms is accompanied by a much smaller secondary concentration around 14–16 ms. The 95th and 99th percentile inter-arrival times were 14.5 ms and 15.8 ms respectively, confirming the presence of intermittent host-side stalls.

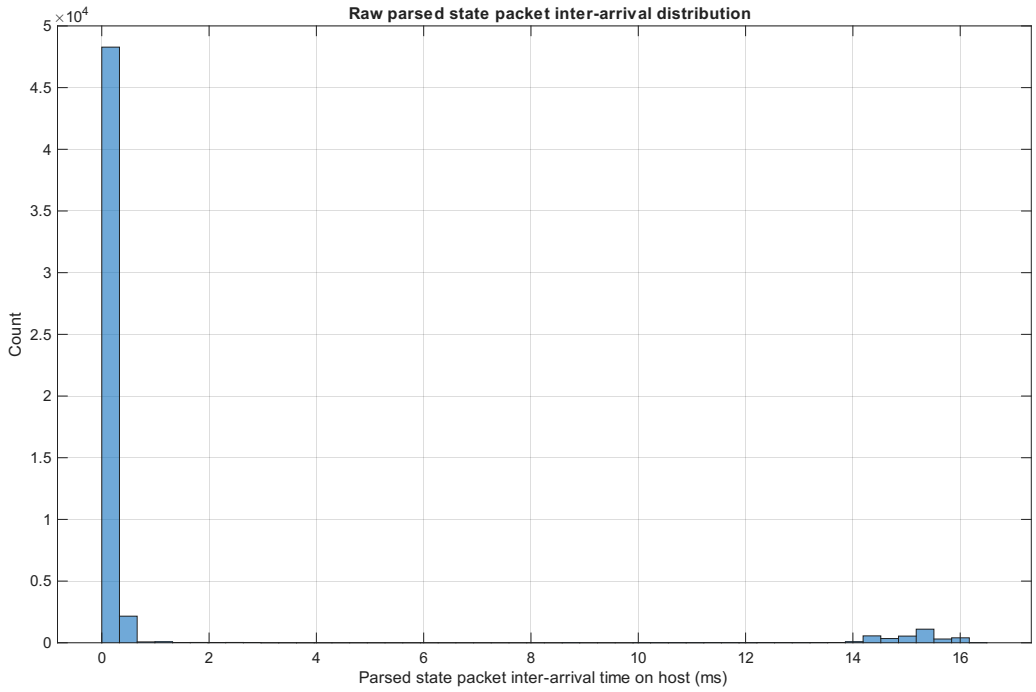


Fig. 9. Distribution of host-side inter-arrival time between consecutively parsed state packets. A dominant peak near 0 ms and a smaller concentration around 14–16 ms indicate burst-like packet processing at the host, consistent with buffered delivery and intermittent scheduling delays.

Despite this non-uniform host-side processing pattern, analysis of MCU-side timestamps showed that the embedded system maintained a stable and consistent update rate. The effective MCU period per state was approximately 1.00 ms, corresponding to an update rate of 1000 Hz, with very low variability.

Taken together, packet generation on the embedded controller was highly regular, while host-side packet processing was non-uniform and burst-like. Despite this, host-side processing latency remained low.

6.2 Simulation Validation Results

Simulation validation tests were conducted with actuator output disabled, allowing the behaviour of the haptic rendering model to be evaluated independently of hardware dynamics and communication effects.

6.2.1 Virtual Wall Model Response

A virtual wall test was performed to evaluate the output forces generated by the haptic rendering engine in isolation from physical feedback.

The rendered force exhibited the expected unilateral contact behaviour. No significant force was produced when the end-effector remained outside the virtual boundary, while penetration resulted in a restoring force that increased with penetration depth, consistent with the implemented spring–damper virtual coupling model. This behaviour matches the expected response of a unilateral contact model, in which force is applied only during penetration and acts along the surface normal.

However, this behaviour degraded under conditions of large penetration. In these cases, both the magnitude and direction of the rendered force became inconsistent. This is attributed to limitations of the projection-based contact model, where the closest-point computation becomes less reliable at greater distances from the surface. As a result, discontinuities in the estimated surface normal and projection point can arise, leading to reduced or misdirected restoring forces.

This behaviour is consistent with known limitations of simplified proxy-based implementations, in which the proxy is directly projected from the current tool position without enforcing continuous constraint satisfaction across timesteps.

This limitation is particularly relevant for haptic stability, as discontinuities in the estimated contact normal can introduce non-physical force variations, degrading perceived stiffness and smoothness during interaction.

Overall, this test confirms that the contact model produces physically consistent behaviour under typical interaction conditions, while highlighting limitations under extreme penetration scenarios.

6.2.2 Constrained Motion Model Behaviour

The behaviour of constrained motion was also evaluated with actuator output disabled.

When the end-effector was moved along the virtual wall surface, the computed force remained predominantly normal to the surface, resisting motion into the wall while allowing tangential motion. This behaviour is characteristic of virtual fixture or constraint-based haptic rendering, where motion is restricted in selected directions while remaining unconstrained in others. Such behaviour

is particularly relevant for training applications, where users must learn to follow constrained paths or avoid forbidden regions.

These results demonstrate that the simulation model can reliably generate constraint forces aligned with surface geometry, supporting its suitability for guidance and training-based interaction tasks.

6.3 Safety Validation Results

Safety mechanisms were evaluated primarily as functional pass/fail protections.

6.3.1 Torque Saturation and Rate Limiting

The combined torque saturation and rate limiting mechanisms were effective in maintaining stable and safe actuator behaviour.

Under conditions where large forces were generated by the haptic rendering engine, the host-side force clamp limited the magnitude of commanded forces before transmission. This was further reinforced by an embedded torque clamp, ensuring that actuator outputs remained within predefined safety bounds.

In addition, the implemented slew-rate limiter on the microcontroller significantly improved stability during interaction. Rapid changes in commanded torque were smoothed into gradual transitions, reducing high-frequency oscillations and improving the perceived stability of contact.

Without rate limiting, abrupt force changes resulted in noticeable vibration and unstable behaviour. The inclusion of the rate limiter reduced these effects and contributed to stable sustained interaction.

Overall, force clamping and torque rate limiting maintained bounded and stable actuator behaviour in the tested conditions.

6.3.2 Watchdog Timeout Behaviour

The watchdog was triggered consistently when command updates were absent for longer than the configured timeout interval of X ms.

When the stream of incoming torque commands from the host was interrupted, the embedded controller detected the absence of updates and reset actuator torque commands to zero after the predefined timeout interval.

This resulted in the device returning to a passive, unpowered state without exhibiting unintended motion or sustained force output.

The watchdog behaviour was observed to be consistent and repeatable across multiple trials, providing a robust safeguard against communication failure or host-side faults.

These results confirm effective fail-safe behaviour during communication disruption.

6.4 Hardware Validation Results

Hardware validation focused on qualitative assessment of responsiveness, stability, and perceived interaction behaviour during representative haptic tasks. Direct end-effector force measurement was not available in the prototype, so hardware evaluation was based primarily on observed stability and perceived interaction behaviour rather than absolute force accuracy.

In addition, actuator control was implemented using field-oriented control without current sensing, meaning that commanded torque could not be treated as a precise or directly calibrated measure of physical output torque. The device therefore conveyed qualitative variations in force and approximate relative magnitude, rather than accurately scaled physical force output.

It should also be noted that, although the device was designed as a 2-DOF interface and both joints remained available for device-to-host position sensing, final force-feedback testing was conducted with actuation available only on the first joint. This was due to the failure of one actuator during development, which could not be replaced within the project budget. As a result, the hardware results reported here reflect partial force-feedback operation of the intended 2-DOF architecture rather than full two-axis haptic rendering.

6.4.1 Actuator Torque Response Characterisation

Standalone testing of individual actuators demonstrated that the motor and control hardware were capable of producing responsive and stable torque output.

Step changes in commanded torque resulted in immediate and clearly observable changes in actuator behaviour. Although the actuator control did not provide a calibrated one-to-one relationship between commanded and physical torque, the response was generally smooth and repeatable, with limited overshoot and no sustained oscillatory instability under typical operating conditions.

Some minor oscillations were observed at higher torque levels or during rapid command changes; however, these were mitigated through the use of torque rate limiting and appropriate control parameter selection.

Overall, the qualitative assessment indicated that the actuator response was sufficiently fast and stable for use in haptic rendering applications, where timely and predictable force output is required.

6.4.2 Stable Contact Behaviour (Virtual Wall Interaction)

This test represents the physical realisation of the virtual wall interaction previously validated in simulation.

The integrated system was able to support sustained contact interaction without loss of stability.

During testing, continuous contact with a virtual surface could be maintained for extended periods without runaway motion or divergence. The rendered force remained consistent, and the interaction was perceived as stable and controllable.

Some low-level vibration and noise were present during contact, particularly under high stiffness settings. However, this did not lead to instability and was reduced through the combined use of damping and torque rate limiting.

The system remained responsive to user input while maintaining a stable resistive force, indicating that the closed-loop architecture is suitable for practical haptic interaction scenarios.

These results demonstrate stable contact interaction under real-time conditions despite communication variability.

It is noted that the large penetration effects observed during simulation validation were less prominent during physical interaction. In practice, the rendered contact force limits penetration depth by opposing user motion. However, due to the finite torque output of the actuators and the achievable virtual stiffness, some penetration still occurs during contact.

This effect is more pronounced with small or thin virtual objects, where small physical displacements can correspond to significant virtual penetration.

Although stable contact behaviour was achieved, the rendered force did not exhibit perfectly rigid characteristics. The interaction was observed to have a degree of compliance, with force increasing gradually rather than instantaneously. This behaviour reflects limitations in actuator torque capability, control fidelity, and transmission behaviour, which are discussed further in Section 7.3.

Under attempts to push strongly through a rigidly rendered object, transmission non-idealities were also observed at the base joint. In particular, belt slip occurred under sufficiently high load, leading to a reduction in perceived resistance and a loss of consistency in the rendered force during extreme contact conditions. Although this behaviour was not dominant during typical interaction, it became apparent during aggressive push-through attempts and further limited the maximum practically renderable stiffness.

6.4.3 Stable Contact Behaviour (Constrained Motion)

Constrained motion behaviour was also evaluated under physical operation of the device. Given that physical actuation was available only on the first joint during final testing, the constrained-motion behaviour observed in hardware represents a partial implementation of the intended two-axis guidance behaviour.

When the end-effector was brought into contact with a virtual surface and moved along it, the system generated forces predominantly normal to the surface, resisting penetration while allowing motion tangential to the constraint. This resulted in guidance-like behaviour consistent with the constrained motion model validated in simulation.

An additional observation was made when the device was released near a virtual surface. When positioned close to the surface, the end-effector could remain supported by the rendered contact force, behaving similarly to a physical object resting against a rigid boundary. In contrast, when released further from the surface, the end-effector moved freely until contact was re-established, at which point motion became constrained along the surface.

This behaviour demonstrates that the system is capable of producing physically intuitive constraint forces, while also reflecting the finite stiffness and force limits of the actuator system. The transition between free motion and constrained interaction was smooth and did not result in instability.

The guidance behaviour was effective but not perfectly rigid, with some compliance and variation in constraint force observed during motion. This was consistent with the limited actuator stiffness and partial actuation available in the final hardware configuration.

These results confirm stable and intuitive constraint interaction under real-time conditions.

6.5 Summary of Results

Overall, the system achieved stable real-time interaction on a low-cost platform.

The main measured constraints were fixed communication latency (approximately 15–16 ms), finite actuator capability, partial actuation in final testing, and transmission non-idealities under high load.

Within these constraints, the tests showed low packet loss at high update rates, low host-side processing latency, consistent safety behaviour, and stable qualitative contact/constraint interaction.

7 Discussion

7.1 Communication and Latency Constraints

The experimental results presented in Section 6.1 demonstrate that system performance is fundamentally constrained by communication latency rather than computational limitations. Although the system achieves high update rates of approximately 1 kHz and low host-side processing latency (sub-millisecond, as shown in Section 6.1.3), the measured round-trip communication delay of approximately 15–16 ms (Section 6.1.2) dominates the closed-loop response.

In haptic rendering systems, achievable virtual stiffness is strongly limited by loop delay. Prior work typically assumes servo update rates on the order of 1 kHz with minimal latency in order to render stiff contact interactions. However, the results presented here show that even when high update rates are maintained, the presence of significant communication delay imposes a practical upper bound on stable stiffness. Given a nominal update period of approximately 1 ms, the measured round-trip delay is more than an order of magnitude larger than the control timestep, introducing substantial phase lag into the feedback loop.

This limitation arises because delayed force feedback introduces phase lag into the closed-loop system, reducing stability margins and increasing the risk of oscillatory behaviour during contact. As a result, the system cannot safely render very high stiffness values without additional stabilisation mechanisms.

Nevertheless, the results demonstrate that stable haptic interaction remains achievable despite this latency, provided that appropriate architectural and control strategies are employed. This reflects a practical distinction between theoretical haptic rendering assumptions and real-world system constraints, where communication delays are often unavoidable.

A likely contributing factor to the observed communication delay is the use of the ST-Link virtual COM interface for host–device communication. This interface was selected due to its simplicity and ease of integration during development, allowing rapid prototyping without requiring additional hardware or low-level USB implementation. However, the burst-like packet delivery observed in Section 6.1.3, together with the approximately 15–16 ms round-trip latency, suggests that this interface introduces buffering and scheduling overhead that is significant relative to the 1 kHz control timestep.

While the present work does not isolate the exact contribution of each component in the communication stack, it is likely that a substantial portion of the measured latency arises from the virtual COM transport layer rather than host-side computation or control loop execution. Alternative communication approaches, such as direct UART communication via a dedicated USB–UART bridge or lower-latency interfaces, could therefore potentially reduce the round-trip delay and improve achievable haptic stiffness.

These results highlight that, in practical haptic systems, communication architecture can be a dominant performance constraint, and must be considered alongside control and mechanical design when targeting high-fidelity interaction.

7.2 Stability of the Haptic Interaction

Despite the presence of significant communication latency and non-uniform packet arrival behaviour observed in Section 6.1.3, the system was able to maintain stable haptic interaction under a wide range of operating conditions. This stability can be attributed to several key design choices in the system architecture.

Firstly, the use of snapshot-based communication channels ensures that the haptic rendering loop always operates on the most recent available device state, rather than processing queued historical data. This effectively mitigates the impact of burst-like packet delivery observed in the communication results, preventing outdated state information from introducing additional delay or instability into the control loop.

Secondly, the separation of communication, haptic rendering, and physics simulation into independent threads allows each subsystem to operate at an appropriate rate without blocking time-critical operations. This decoupling reduces the propagation of timing variability between subsystems.

In addition, stability was significantly improved through the use of damping and torque rate limiting at the embedded control level, as evidenced by the reduction in oscillatory behaviour observed during hardware validation (Section 6.3.1). The virtual coupling model introduces dissipative behaviour that helps to counteract the destabilising effects of delayed feedback, while the slew-rate limiter constrains rapid changes in commanded torque. This prevents abrupt force discontinuities that could otherwise excite oscillations in the system.

Together, these mechanisms enable the system to maintain stable and controllable interaction despite communication delays that would otherwise limit performance in a purely high-stiffness haptic rendering framework.

7.3 Hardware Limitations

The performance and evaluation of the system were subject to several hardware-related limitations arising from practical integration challenges and resource constraints during development. These limitations are directly reflected in the qualitative interaction behaviour observed during hardware validation (Section 6.4), including the presence of compliant contact behaviour and variation in rendered force.

A primary limitation was the absence of current sensing in the actuator control system. The initial design intended to utilise integrated motor driver solutions based on the STM32 B-G431-ESC1 platform, which provide built-in current sensing capabilities suitable for accurate torque control. However, significant integration challenges were encountered when attempting to interface these drivers with the selected magnetic encoders, particularly in achieving reliable and stable operation within the required control framework.

Due to the increased implementation complexity and project time constraints, a design decision was made to adopt SimpleFOC Mini driver modules, which provided a more straightforward and reliable integration pathway with the STM32 Nucleo development board. This allowed development effort to be focused on the embedded control and host-side software architecture, which formed a central objective of the project.

However, the use of SimpleFOC Mini drivers without current sensing introduces an important limitation: commanded torque cannot be interpreted as a calibrated physical torque output. Instead, torque control operates effectively open-loop with respect to current, making the output dependent on motor characteristics, supply conditions, and load variations. As a result, the system provides qualitative rather than quantitatively accurate force feedback. Despite this limitation, the force feedback was sufficient to convey relative interaction forces and contact conditions to the user.

A second limitation arises from the mechanical transmission used to increase actuator torque. A GT2 timing belt reduction was implemented on the base joint to compensate for the limited torque output of the selected low-cost gimbal motors. While effective in increasing available torque, this transmission introduced non-ideal behaviour under high load conditions. During interaction with large rendered forces, the belt was observed to slip when the applied load exceeded the transmission capability. An adjustable idler pulley was introduced to increase belt tension and reduce slip; however, this did not fully eliminate the issue. Residual slip is likely due to the use of 3D-printed pulleys, which do not achieve the same tooth profile precision and engagement as manufactured components.

In addition, the overall torque capability of the selected motors was found to be insufficient for rendering very stiff virtual environments. Even in the absence of belt slip, the actuators were unable to generate sufficiently large torques to convincingly simulate hard contact conditions. As observed in Section 6.4.2, this results in noticeable compliance during interaction with virtual walls, where force increases gradually rather than instantaneously.

A further limitation arose from the failure of one actuator during development. The exact cause of this failure could not be determined, and the actuator was not replaced within the available project timeframe and budget. Consequently, final hardware validation was conducted using a single actuated joint, while the second joint remained available for position sensing only, as described in Section 6.4. Although this did not affect the system architecture or communication framework, it reduced the system from full two-degree-of-freedom force feedback to partial actuation during

physical testing.

Overall, these results demonstrate that, in addition to communication latency, actuator capability and transmission design impose significant constraints on achievable haptic performance in low-cost systems.

7.4 Design Trade-Offs

The design of the system reflects a set of trade-offs between stability, responsiveness, and achievable force fidelity.

While the use of damping and torque rate limiting was effective in ensuring stable interaction, these mechanisms introduce a reduction in perceived stiffness and responsiveness. In particular, the rate limiting of actuator torque results in a finite rise time when contact forces are applied. This leads to a short delay between initial contact and the development of the full resistive force, which reduces the sharpness of hard contact interactions such as collisions with rigid virtual walls, as observed during hardware validation (Section 6.4.2).

As a result, high-speed interactions and abrupt contacts are not rendered with the same fidelity as would be expected in systems with lower latency and higher achievable stiffness. Instead, contact forces are perceived as gradually increasing rather than instantaneous, which can affect realism in scenarios involving rigid constraints.

However, this trade-off is necessary to ensure stable behaviour in the presence of communication delay. Without rate limiting and damping, the system exhibited oscillatory and unstable behaviour during contact, particularly under high stiffness settings, as demonstrated in Section 6.3.1. The implemented approach therefore prioritises stability and controllability over maximum stiffness, which is appropriate for a low-cost prototype system.

These results highlight a key design consideration in practical haptic systems: achieving stable interaction under real-world constraints often requires compromising on ideal force rendering performance.

8 Conclusions and Future Work

8.1 Conclusions

This project presented the design, implementation, and evaluation of a low-cost, modular haptic rendering system integrating a two-degree-of-freedom input device with a custom real-time simulation and visualisation framework.

The system successfully achieved its primary objective of enabling bidirectional interaction between a user-operated device and a virtual environment, with real-time computation of interaction forces and feedback through actuator torque control. The architecture was designed to prioritise modularity and extensibility, allowing clear separation between hardware, communication, simulation, and rendering subsystems.

Experimental results demonstrated that the system is capable of sustaining high update rates of approximately 1 kHz, with negligible packet loss at higher transmission frequencies and low host-side processing latency. However, communication analysis revealed a consistent round-trip delay of approximately 15–16 ms, which dominates the closed-loop response and imposes a fundamental limitation on achievable haptic performance.

Despite this latency, stable and controllable haptic interaction was achieved through a combination of architectural and control strategies. The use of snapshot-based communication ensured that the system operated on the most recent available data, mitigating the effects of burst-like packet delivery. In addition, the application of damping and torque rate limiting was critical in maintaining stability, preventing oscillatory behaviour during contact, and enabling sustained interaction with virtual constraints.

Simulation and hardware validation confirmed that the system is capable of rendering physically consistent contact and constraint forces. The virtual coupling and proxy-based rendering approach produced expected unilateral contact behaviour and supported constrained motion along virtual surfaces. Hardware testing further demonstrated that stable interaction could be maintained under real-time conditions, even with communication variability and partial actuation.

However, the results also highlight an important trade-off between stability and force fidelity. While the system remains stable under delayed feedback, the achievable virtual stiffness is limited, and

contact forces exhibit a finite rise time rather than instantaneous response. This reduces the realism of high-speed or high-stiffness interactions, particularly for rigid contact scenarios.

Overall, the project demonstrates that stable and usable haptic interaction can be achieved using low-cost hardware and a modular software architecture, even in the presence of significant communication latency. The findings emphasise that in practical systems, communication delay is often the dominant constraint, and that appropriate architectural and control design is essential to achieving stable performance.

8.2 Future Work

Several avenues for future development have been identified to improve system performance, fidelity, and extensibility.

A key priority is the reduction of communication latency. The current system relies on the ST-Link virtual COM interface, which was selected for its simplicity and ease of integration during early development. While this approach enabled rapid prototyping, the results presented in Section 6.1 indicate that the communication pathway introduces buffering and scheduling delays that dominate the closed-loop response.

Future work should therefore investigate lower-latency communication architectures between the host and embedded controller. In particular, direct UART communication via a dedicated USB–UART bridge may reduce buffering effects associated with the virtual COM interface, while alternative approaches such as native USB implementations (e.g., bulk transfer), CAN bus, or Ethernet-based communication could provide more deterministic timing behaviour.

A systematic comparison of these communication approaches would allow the contribution of each component in the communication stack to be isolated and could provide a pathway toward significantly reducing round-trip latency and improving achievable haptic stiffness.

Improving actuator performance and force fidelity is another critical area. The incorporation of current sensing would enable accurate torque control and allow quantitative validation of rendered forces. This would also support the implementation of more advanced control strategies, including impedance control and operational-space control formulations.

Mechanical transmission design could also be improved. While GT2 timing belts provided a simple and low-cost solution for torque amplification, they introduced slip and compliance under high

load conditions. Future designs could explore alternative transmission mechanisms such as capstan drives, as demonstrated in low-cost haptic systems such as the Hapkit platform. These approaches offer reduced backlash and improved force transmission characteristics, which could enhance both stability and realism.

The use of higher torque actuators or geared motor systems would further improve the ability to render stiff virtual environments, reducing compliance during contact and enabling more realistic hard-contact interactions.

Restoration of full two-degree-of-freedom actuation is also an important step. Enabling force feedback across both joints would improve interaction realism and allow more complex manipulation and constraint-based tasks to be explored.

Further improvements to the haptic rendering model could address limitations observed under large penetration conditions. More advanced contact handling approaches, such as continuous collision detection, constraint-based solvers, or improved normal estimation techniques, could enhance robustness and force consistency.

From a software perspective, the modular architecture provides a strong foundation for extension toward more advanced simulation environments. Integration with external platforms such as NVIDIA Isaac Sim or MATLAB/Simulink would enable more realistic robotic scenarios and facilitate comparison with established robotic control frameworks.

Finally, a more comprehensive evaluation framework incorporating quantitative force measurement, user studies, and task-based performance metrics would provide deeper insight into the effectiveness of the system as a training tool.

Together, these developments would enable the system to evolve from a low-cost prototype into a more capable and general-purpose haptic interface platform for robotic interaction and training applications.

References

[1] G. S. Giri, Y. Maddahi, and K. Zareinia, “An Application-Based Review of Haptics Technology,” *Robotics*, vol. 10, no. 1, p. 29, Feb. 5, 2021, ISSN: 2218-6581. DOI: 10.3390/robotics10010029 Accessed: Apr. 16, 2026. [Online]. Available: https://www.mdpi.com/2218-6581/10/1/29 (cited on pp. 1, 4, 6).

[2] M. Bergholz, M. Ferle, and B. M. Weber, “The benefits of haptic feedback in robot assisted surgery and their moderators: A meta-analysis,” *Scientific Reports*, vol. 13, no. 1, p. 19 215, Nov. 6, 2023, ISSN: 2045-2322. DOI: 10.1038/s41598-023-46641-8 Accessed: Apr. 16, 2026. [Online]. Available: https://www.nature.com/articles/s41598-023-46641-8 (cited on pp. 1, 4).

[3] D. C. Ruspini, K. Kolarov, and O. Khatib, “The haptic display of complex graphical environments,” in *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques - SIGGRAPH ’97*, Not Known: ACM Press, 1997, pp. 345–352, ISBN: 978-0-89791-896-1. DOI: 10.1145/258734.258878 Accessed: Apr. 16, 2026. [Online]. Available: http://portal.acm.org/citation.cfm?doid=258734.258878 (cited on pp. 1, 5).

[4] R. Adams and B. Hannaford, “Stable haptic interaction with virtual environments,” *IEEE Transactions on Robotics and Automation*, vol. 15, no. 3, pp. 465–474, Jun. 1999, ISSN: 1042296X. DOI: 10.1109/70.768179 Accessed: Apr. 16, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/768179/ (cited on pp. 1, 5, 13).

[5] M. O. Martinez et al., “3-D printed haptic devices for educational applications,” in *2016 IEEE Haptics Symposium (HAPTICS)*, Apr. 2016, pp. 126–133. DOI: 10.1109/HAPTICS.2016.7463166 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/7463166 (cited on pp. 2, 6).

[6] M. O. Martinez et al., “Open source, modular, customizable, 3-D printed kinesthetic haptic devices,” in *2017 IEEE World Haptics Conference (WHC)*, Jun. 2017, pp. 142–147. DOI: 10.1109/WHC.2017.7989891 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7989891 (cited on pp. 2, 6).

[7] “Inverse3 X,” Accessed: Apr. 16, 2026. [Online]. Available: https://www.haply.co/inverse3x (cited on p. 5).

[8] “Geomagic touch haptic device,” Accessed: Apr. 16, 2026. [Online]. Available: https://shop.gomeasure3d.com/products/geomagic-touch-haptic-device (cited on p. 5).

[9] "Geomagic touch X haptic device," Accessed: Apr. 16, 2026. [Online]. Available: https://shop.gomeasure3d.com/products/geomagic-touch-x-haptic-device (cited on p. 5).

[10] C. Zilles and J. Salisbury, "A constraint-based god-object method for haptic display," in *Proceedings 1995 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human Robot Interaction and Cooperative Robots*, vol. 3, Pittsburgh, PA, USA: IEEE Comput. Soc. Press, 1995, pp. 146–151, ISBN: 978-0-8186-7108-1. DOI: 10.1109/IROS.1995.525876 Accessed: Apr. 16, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/525876/ (cited on p. 5).

[11] A. Lavatelli, M. Bordegoni, and F. Ferrise, "Design of an open-source low cost 2DOF haptic device," Sep. 2014. DOI: 10.1109/MESA.2014.6935574 (cited on p. 6).

[12] A. Skuric et al., "SimpleFOC: A field oriented control (FOC) library for controlling brushless direct current (BLDC) and stepper motors," *Journal of Open Source Software*, vol. 7, no. 74, p. 4232, 2022. DOI: 10.21105/joss.04232 [Online]. Available: https://doi.org/10.21105/joss.04232 (cited on p. 10).

[13] P. Vas, *Sensorless Vector and Direct Torque Control*. Oxford University Press, May 14, 1998, ISBN: 978-0-19-856465-2. DOI: 10.1093/oso/9780198564652.001.0001 Accessed: Apr. 16, 2026. [Online]. Available: https://doi.org/10.1093/oso/9780198564652.001.0001 (cited on p. 10).

[14] C. Ericson, *Real-Time Collision Detection*, 0th ed. CRC Press, Dec. 22, 2004, ISBN: 978-0-08-047414-4. DOI: 10.1201/b14581 Accessed: Apr. 16, 2026. [Online]. Available: https://www.taylorfrancis.com/books/9780080474144 (cited on p. 13).

Appendices

A Project outline

Project outline as submitted at the start of the project is a required appendix. Put here.

B Risk assessment

Risk assessment is a required appendix. Put here.

C Code